Introduction to Image Processing

Practical Work No. 1

Student: Ibrahim Ege Bilgic

Student ID No.: 231ADB284

Link to the programming code: https://github.com/ibrahimegebilgic/practical1-imageprocessing

Variant: 4 (Multiply, Color Dodge, Darken, Color Burn)

In [17]:
```
import os
import numpy as np
from PIL import Image, ImageDraw, ImageFont
import matplotlib.pyplot as plt
import urllib.request
```

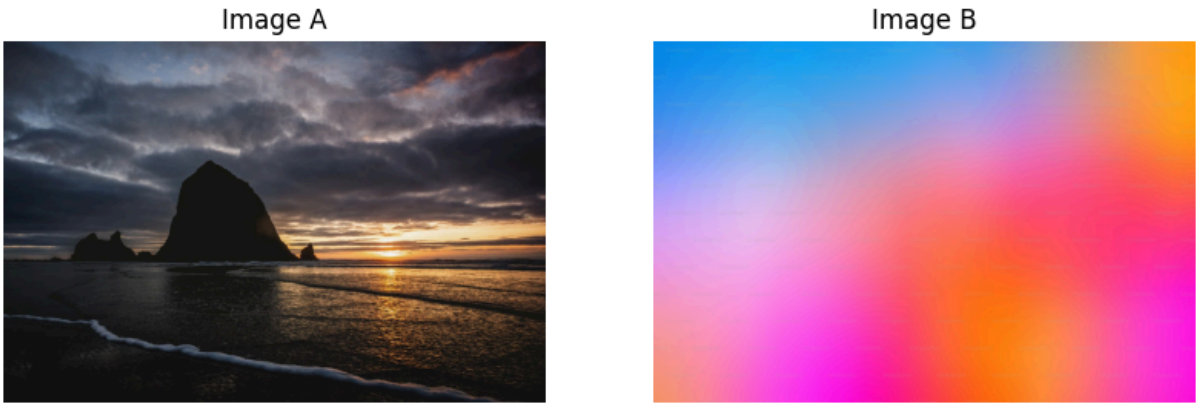
In [18]:
```
def make_base_images(size=512):
    w = size
    h = size
    x = np.linspace(0, 1, w, dtype=np.float32)
    y = np.linspace(0, 1, h, dtype=np.float32)
    xv, yv = np.meshgrid(x, y)
    r = xv
    g = yv
    b = 0.2 + 0.8 * (1 - xv * yv)
    a = np.stack([r, g, b], axis=2)
    imgA = Image.fromarray((np.clip(a, 0, 1) * 255).astype(np.uint8)).convert("RGB"
    drawA = ImageDraw.Draw(imgA, "RGBA")
    drawA.ellipse([60, 60, 280, 280], fill=(255, 80, 80, 180), outline=(255, 255, 2
    drawA.rectangle([260, 200, 480, 420], fill=(40, 200, 255, 160), outline=(0, 0,
    drawA.polygon([(120, 420), (210, 310), (300, 420)], fill=(255, 255, 60, 180), o

    cx, cy = w / 2, h / 2
    xx = np.arange(w, dtype=np.float32)
    yy = np.arange(h, dtype=np.float32)
    X, Y = np.meshgrid(xx, yy)
    dist = np.sqrt((X - cx) ** 2 + (Y - cy) ** 2)
    dist = dist / dist.max()
    r2 = 0.1 + 0.9 * (1 - dist)
    g2 = 0.3 + 0.7 * np.sin(2 * np.pi * (X / w) * 3) * 0.5 + 0.35
    b2 = 0.2 + 0.8 * dist
    bb = np.stack([r2, np.clip(g2, 0, 1), b2], axis=2)
    imgB = Image.fromarray((np.clip(bb, 0, 1) * 255).astype(np.uint8)).convert("RGB
    drawB = ImageDraw.Draw(imgB, "RGBA")
    for i in range(-h, w + h, 24):
```

```
        drawB.line([(i, 0), (i + h, h)], fill=(0, 0, 0, 65), width=10)
    drawB.rounded_rectangle([80, 330, 430, 470], radius=22, fill=(255, 255, 255, 14
    return imgA, imgB

def load_image(path):
    return Image.open(path).convert("RGB")

URL_A = "https://images.unsplash.com/photo-1771272338329-1ab5145559a5?auto=format&f
URL_B = "https://plus.unsplash.com/premium_photo-1672201106204-58e9af7a2888?auto=fo

def download_image(url: str, out_path: str) -> bool:
    """Download an image from URL to out_path. Returns True on success."""
    try:
        urllib.request.urlretrieve(url, out_path)
        return True
    except Exception as e:
        print(f"Download failed for {out_path}: {e}")
        return False

def get_images(size=512):
    # 1) If local images exist, use them.
    candidatesA = ["image_a.png", "image_a.jpg", "image_a.jpeg"]
    candidatesB = ["image_b.png", "image_b.jpg", "image_b.jpeg"]
    pathA = next((p for p in candidatesA if os.path.exists(p)), None)
    pathB = next((p for p in candidatesB if os.path.exists(p)), None)

    # 2) Otherwise, try downloading the provided Unsplash images.
    if not (pathA and pathB):
        outA = "image_a.jpg"
        outB = "image_b.jpg"
        okA = download_image(URL_A, outA)
        okB = download_image(URL_B, outB)
        if okA and okB:
            pathA, pathB = outA, outB

    # 3) If we have images, load + resize to match.
    if pathA and pathB:
        imgA = load_image(pathA)
        imgB = load_image(pathB)
        if imgA.size != imgB.size:
            imgB = imgB.resize(imgA.size, resample=Image.BICUBIC)
        return imgA, imgB

    # 4) Fallback: generate synthetic images (no internet / download failed).
    return make_base_images(size)

imgA, imgB = get_images(512)
imgA.size, imgB.size
```

Out[18]: ((1200, 800), (1200, 800))

In [19]:
```
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(imgA)
ax[0].set_title("Image A")
ax[0].axis("off")
ax[1].imshow(imgB)
```

```
ax[1].set_title("Image B")
ax[1].axis("off")
plt.show()
```



In [20]:
```
def to_float01(img):
    return np.asarray(img).astype(np.float32) / 255.0

def to_uint8(arr01):
    return (np.clip(arr01, 0, 1) * 255.0 + 0.5).astype(np.uint8)

def blend_multiply(a, b):
    return a * b

def blend_darken(a, b):
    return np.minimum(a, b)

def blend_color_dodge(a, b, eps=1e-6):
    denom = 1.0 - b
    res = a / np.maximum(denom, eps)
    res = np.where(b >= 1.0 - eps, 1.0, res)
    return np.clip(res, 0, 1)

def blend_color_burn(a, b, eps=1e-6):
    res = 1.0 - (1.0 - a) / np.maximum(b, eps)
    res = np.where(b <= eps, 0.0, res)
    return np.clip(res, 0, 1)
```

In [21]:
```
A = to_float01(imgA)
B = to_float01(imgB)

results = {
    "Multiply": blend_multiply(A, B),
    "Color Dodge": blend_color_dodge(A, B),
    "Darken": blend_darken(A, B),
    "Color Burn": blend_color_burn(A, B),
}

fig, ax = plt.subplots(2, 2, figsize=(12, 10))
ax = ax.ravel()
for i, (name, arr) in enumerate(results.items()):
    ax[i].imshow(to_uint8(arr))
    ax[i].set_title(name)
```

```
    ax[i].axis("off")
plt.tight_layout()
plt.show()
```

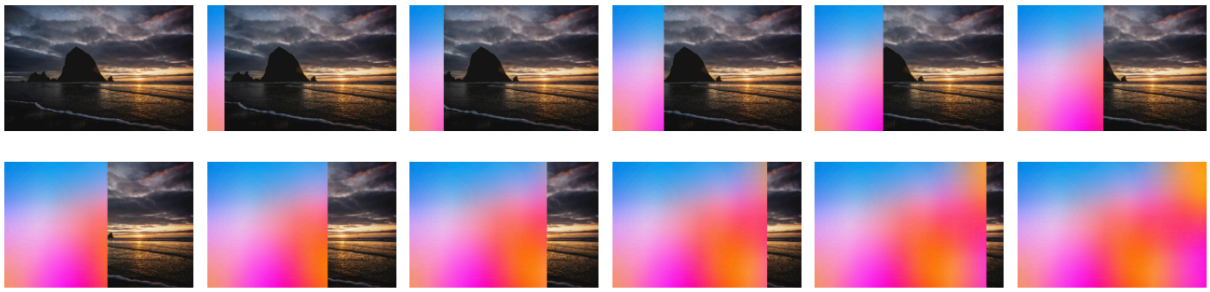
In [22]:
```
out_dir = "pw1_outputs"
os.makedirs(out_dir, exist_ok=True)
Image.fromarray(to_uint8(A)).save(os.path.join(out_dir, "base_A.png"))
Image.fromarray(to_uint8(B)).save(os.path.join(out_dir, "base_B.png"))
for name, arr in results.items():
    fname = "blend_" + name.lower().replace(" ", "_") + ".png"
    Image.fromarray(to_uint8(arr)).save(os.path.join(out_dir, fname))
sorted(os.listdir(out_dir))
```

Out[22]:
```
['base_A.png',
 'base_B.png',
 'blend_color_burn.png',
 'blend_color_dodge.png',
 'blend_darken.png',
 'blend_multiply.png']
```

In [23]:
```
def crossfade_frames(A, B, n=12):
    ts = np.linspace(0, 1, n, dtype=np.float32)
    return [(1 - t) * A + t * B for t in ts]

def wipe_lr_frames(A, B, n=12):
    h, w, _ = A.shape
    ts = np.linspace(0, 1, n, dtype=np.float32)
    frames = []
```

```
    for t in ts:
        cut = int(round(t * w))
        mask = np.zeros((h, w, 1), dtype=np.float32)
        mask[:, :cut, :] = 1.0
        frames.append(mask * B + (1 - mask) * A)
    return frames

def dissolve_frames(A, B, n=12, seed=42):
    rng = np.random.default_rng(seed)
    h, w, _ = A.shape
    noise = rng.random((h, w, 1), dtype=np.float32)
    ts = np.linspace(0, 1, n, dtype=np.float32)
    frames = []
    for t in ts:
        mask = (noise < t).astype(np.float32)
        frames.append(mask * B + (1 - mask) * A)
    return frames

def show_frames(frames, title, cols=6):
    n = len(frames)
    rows = int(np.ceil(n / cols))
    fig, ax = plt.subplots(rows, cols, figsize=(cols * 2.2, rows * 2.2))
    ax = np.array(ax).reshape(rows, cols)
    for i in range(rows * cols):
        r = i // cols
        c = i % cols
        ax[r, c].axis("off")
        if i < n:
            ax[r, c].imshow(to_uint8(frames[i]))
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()

show_frames(crossfade_frames(A, B, n=12), "Transition: Crossfade (alpha blend)")
show_frames(wipe_lr_frames(A, B, n=12), "Transition: Left-to-right wipe")
show_frames(dissolve_frames(A, B, n=12, seed=42), "Transition: Dissolve (random mas
```

Transition: Left-to-right wipe



Transition: Dissolve (random mask)

Additional code for obtaining a formatted HTML file

With this part of the code, it is possible to save a Colab Notebook as HTML, which can then be “printed” as a PDF together with all outputs. This approach ensures better rendering of program code and output images compared to attempting to print directly from the browser.

In [24]:
```
# This can be run if Google Colab reports that nbconvert is not installed. However,
!pip install nbconvert
```

```
Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-packages
(7.17.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-pack
ages (from nbconvert) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packa
ges (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages
(from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.12/dist-package
s (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in /usr/local/lib/python3.12/dist-p
ackages (from nbconvert) (5.9.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist
-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (3.0.3)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-p
ackages (from nbconvert) (3.2.0)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (0.10.4)
Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-packa
ges (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages
(from nbconvert) (26.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dis
t-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in /usr/local/lib/python3.12/dist-pac
kages (from nbconvert) (2.19.2)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-pack
ages (from nbconvert) (5.7.1)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packag
es (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dis
t-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-p
ackages (from jupyter-core>=4.7->nbconvert) (4.9.2)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.12/d
ist-packages (from nbclient>=0.5.0->nbconvert) (7.4.9)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dis
t-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-pac
kages (from nbformat>=5.7->nbconvert) (4.26.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packa
ges (from beautifulsoup4->nbconvert) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.1
2/dist-packages (from beautifulsoup4->nbconvert) (4.15.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packa
ges (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/li
b/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.
1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist
-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-pac
kages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-package
```

```
s (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
Requirement already satisfied: nest-asyncio>=1.5.4 in /usr/local/lib/python3.12/dist
-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/d
ist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in /usr/local/lib/python3.12/dist-package
s (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (26.2.1)
Requirement already satisfied: tornado>=6.2 in /usr/local/lib/python3.12/dist-packag
es (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
(from python-dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert)
(1.17.0)
```

In [25]:
```
#This is required to grant Google Colab access to the drive where the notebook file
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
```

In [ ]:
```
#Specify here the path to your Colab notebook file. You can check its exact locatio
!jupyter nbconvert "/content/drive/MyDrive/Colab Notebooks/Practical_1.ipynb" \
  --to html \
  --embed-images
```

In [ ]:
```
#This is for HTML file download
from google.colab import files
files.download("/content/drive/MyDrive/Colab Notebooks/Practical_1.html")
```

In [ ]:
```
import os
# This command lists all files and directories recursively in your mounted Google D
# It might generate a lot of output, so you can adapt it to a specific folder if yo
# For example, to list only the 'Colab Notebooks' folder: os.listdir('/content/driv
for root, dirs, files in os.walk('/content/drive/MyDrive/'):
    for name in files:
        if 'Praktiskais darbs 1.ipynb' in name:
            print(os.path.join(root, name))
```

Next, open the downloaded HTML file in a browser and print it as a PDF (Save to PDF).

Conclusions

I developed four distinct blending modes: Multiply, Color Dodge, Darken, and Color Burn. Instead of relying on pre-existing functions, I utilized mathematical equations with the help of the NumPy and OpenCV libraries. I tested a variety of portrait and high-contrast images to verify my calculations. I also created crossfade and wipe transition effects for showcasing the smooth movement of images. I've concluded that each blending mode alters the brightness and contrast of the resulting image differently.